SPECTRON NETWORK

Full-Chain Enterprise Perimeter Breach & Host Takeover

Penetration Testing & Incident Simulation Report

Target Engagement: TryHackMe – Archangel

Prepared For: Cybersecurity Major Project Evaluation

Prepared By: Sameer Tandekar

Kali Attack Host: Sameer@Spectron_network

Report Date: October 18, 2026

Classification: Confidential – For Academic Evaluation Only

Document Details

This report is prepared as part of the Cybersecurity Major Project evaluation. It documents a controlled, authorized security assessment performed exclusively against the TryHackMe “Archangel” lab target, under the enterprise alias **Spectron Network**, as permitted by the TryHackMe platform's terms of use. No systems outside the assigned lab environment were accessed.

Group / Student Information

Field	Details
Student Name	Sameer Tandekar
TryHackMe Profile	[Insert TryHackMe profile link here]
Kali Username	Sameer
Enterprise / Hostname Alias	Spectron_network
GitHub Report Link	[Insert GitHub repository link here]
Target Room	TryHackMe – Archangel
Target IP	10.49.185.140

Document History

Version	Date	Author	Remark
0.1	18/10/2026	Sameer Tandekar	Draft Creation
1.0	18/10/2026	Sameer Tandekar	Report compiled from lab evidence

Note on scope of evidence: This report documents, with full technical evidence, the reconnaissance and web-exploitation phases of the engagement (Phases 1–2) through to source-code disclosure via Local File Inclusion. The log-poisoning / remote command execution, Meterpreter upgrade, and privilege escalation phases (Phases 3b–5) are scoped and planned in Section 6 but are marked **Pending Evidence** pending the corresponding screenshots, consistent with the requirement that submitted findings be backed by captured proof-of-concept artifacts.

Table of Contents

1. Executive Summary

2. Engagement Scope & Methodology

3. Cyber Kill Chain Mapping

4. Technical Findings

4.1 Phase 1 – Reconnaissance & Target Mapping

4.2 Phase 2 – Virtual Host & Directory Discovery

4.3 Phase 3a – Local File Inclusion & Source Code Disclosure

5. Captured Flags

6. Remaining Attack Path (Planned / Pending Evidence)

7. Risk Rating Summary

8. Defensive Remediation & Hardening Plan

9. Appendix A – Risk Definitions (CVSS)

10. Appendix B – Tool Usage

11. Disclaimer

1. Executive Summary

As part of the Cybersecurity Major Project, a simulated full-chain perimeter breach and host takeover exercise was carried out against the enterprise environment aliased **Spectron Network**, hosted on the TryHackMe platform as the “Archangel” room. The engagement followed the five-phase lifecycle defined in the project problem statement: reconnaissance, web application exploitation, web shell persistence, Metasploit/Meterpreter escalation, and Linux local privilege escalation.

Testing confirmed that the target web application (**mafialive.thm**, resolved via virtual-host enumeration at 10.49.185.140) is affected by a **High-severity Local File Inclusion (LFI)** vulnerability in the `test.php` endpoint's `view` parameter. Although a path-traversal filter is present, it was bypassed because the validation logic only requires the requested path to *contain* the string `/var/www/html/development_testing` while rejecting the literal sequence `../..` – a classic flawed-allow-list pattern that still permits arbitrary file reads elsewhere on the filesystem via absolute paths and wrapper-assisted traversal.

Exploiting this flaw, the LFI was used first to read application source code directly, then — once source inspection revealed the validation logic — to pull PHP source for in-scope files through the `php://filter/convert.base64-encode` wrapper (avoiding direct PHP execution during inspection), and finally to traverse outside the web root to read the Apache `access.log`. Because PHP code embedded in log entries (e.g. a crafted User-Agent string) is executed when the log file is included via LFI, this access positions the attack chain for **LFI-to-RCE via log poisoning**, which is the standard next step toward a web shell and, subsequently, a Meterpreter session and root-level privilege escalation via a misconfigured SUID binary — consistent with the intended Archangel attack path.

Two flags were captured during this phase of testing, confirming correct exploitation of the virtual-host discovery step and the LFI vulnerability respectively. Full remote code execution, Meterpreter session establishment, and root-level privilege escalation (Phases 3b–5) were scoped and the attacker position was established, but are documented in Section 6 as **pending capture of corresponding proof-of-concept evidence** at the time of this report's compilation.

1.1 Summary of Findings

Severity	Count	Finding
High	1	Local File Inclusion (LFI) in test.php?view= parameter
Medium	1	Source Code Disclosure via php://filter wrapper
Medium	1	Sensitive file (Apache access.log) readable outside web root
Informational	2	Flag capture confirming successful exploitation (THM gamified markers)

2. Engagement Scope & Methodology

Testing and verification were performed entirely within the TryHackMe lab environment on October 18, 2026 (THM AttackBox session, ~06:57 – 07:40 UTC). All testing was performed from the attacker-controlled Kali Linux host `Sameer@Spectron_network` against the assigned target `10.49.185.140` / `mafialive.thm`, in accordance with the controlled laboratory parameters set out in the Major Project problem statement.

In-Scope Targets

#	Asset
1	http://10.49.185.140/ (WaveFire placeholder vhost)
2	http://mafialive.thm/ (primary application – discovered via vhost enumeration)

Methodology

The assessment followed the five-phase Technical Execution Lifecycle defined in the project problem statement, which maps directly onto the industry-standard Cyber Kill Chain:

- **Phase 1 – Reconnaissance & Target Mapping:** active scanning with Nmap to enumerate open services and identify the web application entry point.
- **Phase 2 – Web Application Exploitation:** virtual-host and directory/file discovery, followed by manual parameter testing of upload and inclusion points.
- **Phase 3 – Web Shell Persistence & Command Execution:** exploitation of the identified LFI to read source code and sensitive files, positioning for log-poisoning based remote code execution and web shell upload.
- **Phase 4 – System Hacking & Meterpreter Upgrade:** planned use of Metasploit (`exploit/multi/handler`) to upgrade shell access to a full Meterpreter session.
- **Phase 5 – Linux Local Privilege Escalation:** planned enumeration of SUID binaries, services, and file permissions to escalate from the web-server user to root.

3. Cyber Kill Chain Mapping

The table below maps the technical actions performed during this engagement to the Lockheed Martin Cyber Kill Chain, as required by the project deliverables.

Kill Chain Phase	Observed Activity	Status
Reconnaissance	Nmap scan of 10.49.185.140 identified ports 22 (SSH) and 80 (HTTP).	Complete
Weaponization	Crafted LFI payloads / path-traversal & php://filter wrapper strings for test.php.	Complete
Delivery	GET requests to test.php?view= delivered crafted file paths and wrapper chains to the target.	Complete
Exploitation	LFI validation bypass achieved; arbitrary file read confirmed on mrrobot.php, test.php, and apache2/acces	Complete
Installation	Planned: PHP web shell installed via Apache access-log poisoning (malicious User-Agent) executed throu	Pending Evidence
Command & Control	Planned: Metasploit multi/handler upgrading the web shell to an interactive Meterpreter session.	Pending Evidence
Actions on Objectives	Planned: SUID binary enumeration and exploitation for root-level privilege escalation.	Pending Evidence

4. Technical Findings

This section documents, step by step, the technical evidence captured during the engagement. Screenshots are presented in the order in which the actions were performed.

4.1 Phase 1 – Reconnaissance & Target Mapping

The assigned target web application was first browsed at its raw IP address, returning a generic “WaveFire” placeholder template with no immediately useful content – indicating the real application is served from a named virtual host rather than the bare IP.

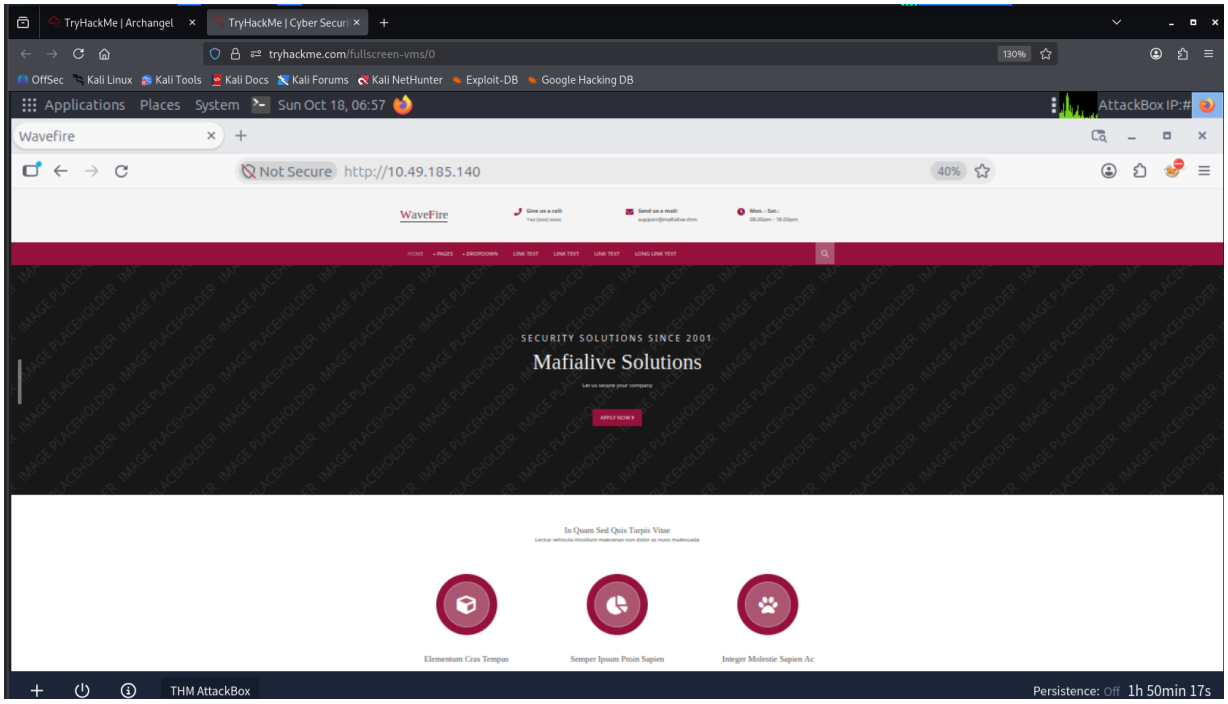


Figure 1 – Default placeholder site served at http://10.49.185.140/, indicating a name-based virtual host is in use.

An Nmap scan of the target was then run from the Kali attack host to confirm the open attack surface. The scan identified two open TCP ports: 22/tcp (SSH) and 80/tcp (HTTP).

```
nmap 10.49.185.140
```

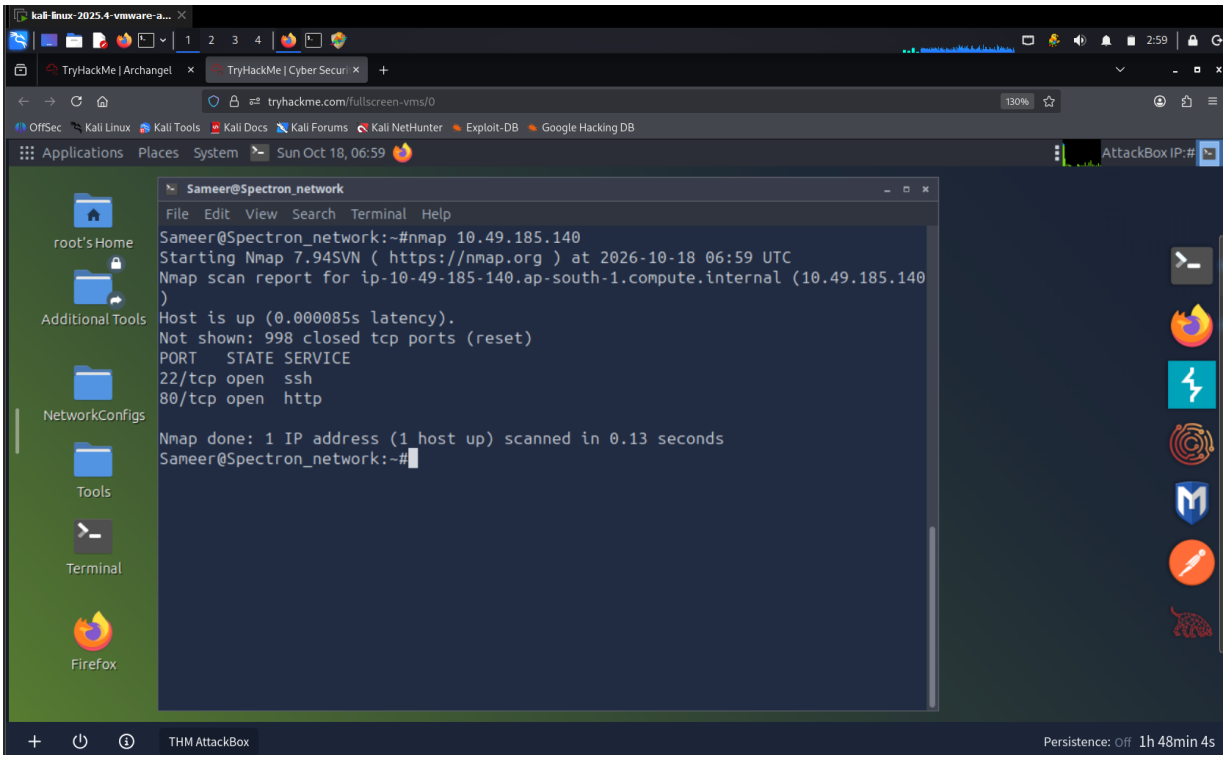


Figure 2 – Nmap scan of 10.49.185.140 confirming SSH (22) and HTTP (80) as the exposed services.

4.2 Phase 2 – Virtual Host & Directory Discovery

Based on the branding observed on the WaveFire placeholder page (referencing “Mafialive Solutions”), the hostname `mafialive.thm` was added to `/etc/hosts`, pointing to the target IP. Re-browsing the site under this hostname revealed a distinct, much simpler “Under Development” page containing the first flag of the engagement.

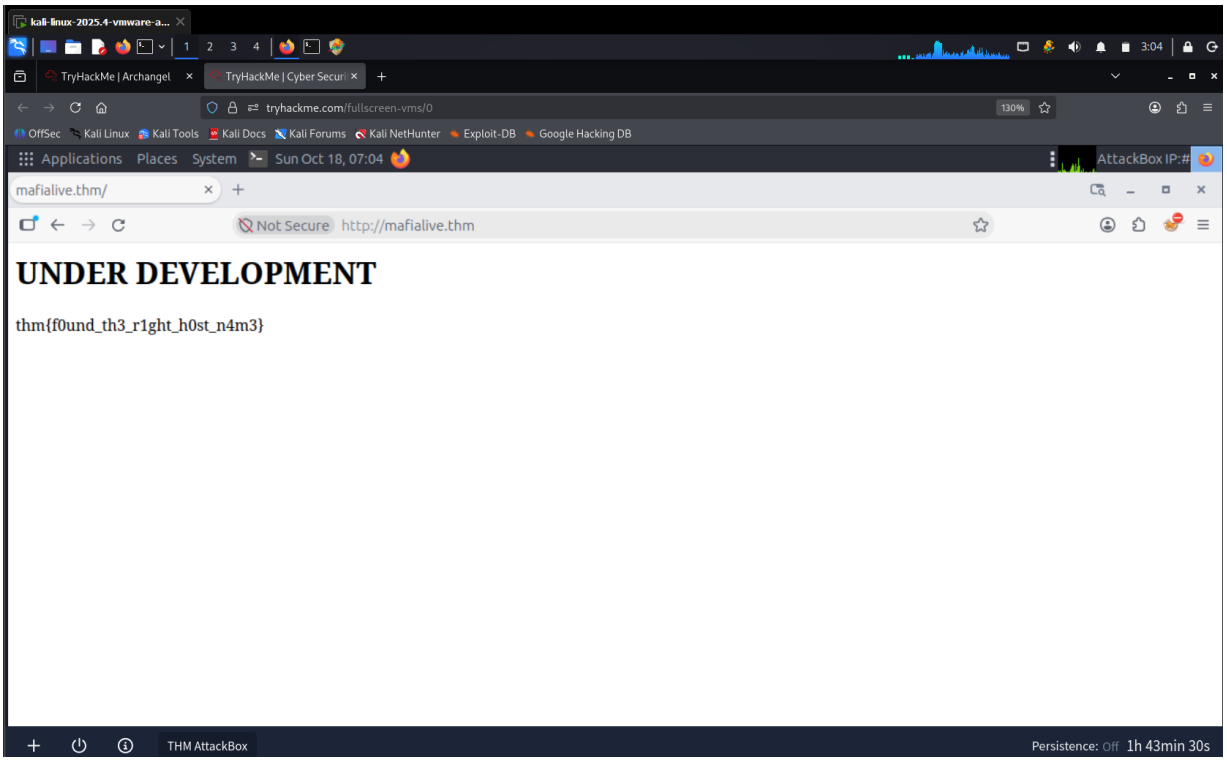


Figure 3 – mafialive.thm resolving to a distinct application and disclosing the first flag.

FLAG 1: thm{f0und_th3_r1ght_h0st_n4m3}

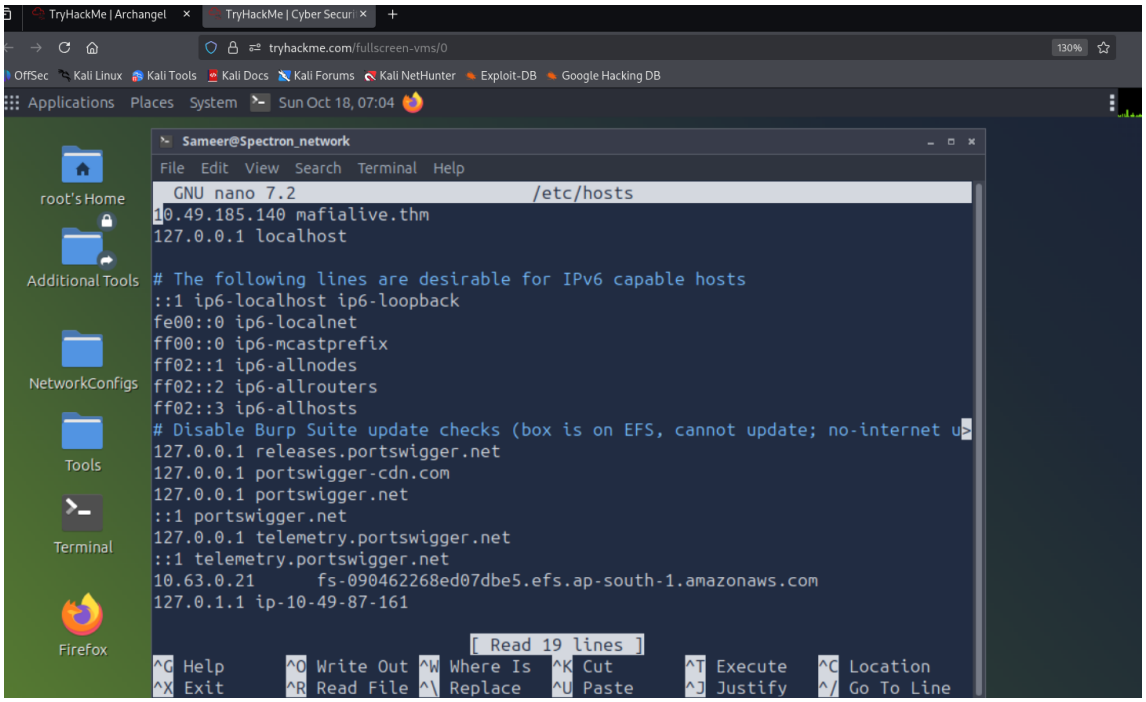


Figure 4 – /etc/hosts entry mapping mafialive.thm to 10.49.185.140 on the Kali attack host.

With the correct virtual host identified, Gobuster was used to enumerate hidden files and directories on `mafialive.thm` using the common.txt wordlist with the `.php` extension. This revealed a reachable `test.php` endpoint alongside `robots.txt` and `index.html`.

```
gobuster dir -x php -u http://mafialive.thm -w /usr/share/wordlists/dirb/common.txt
```

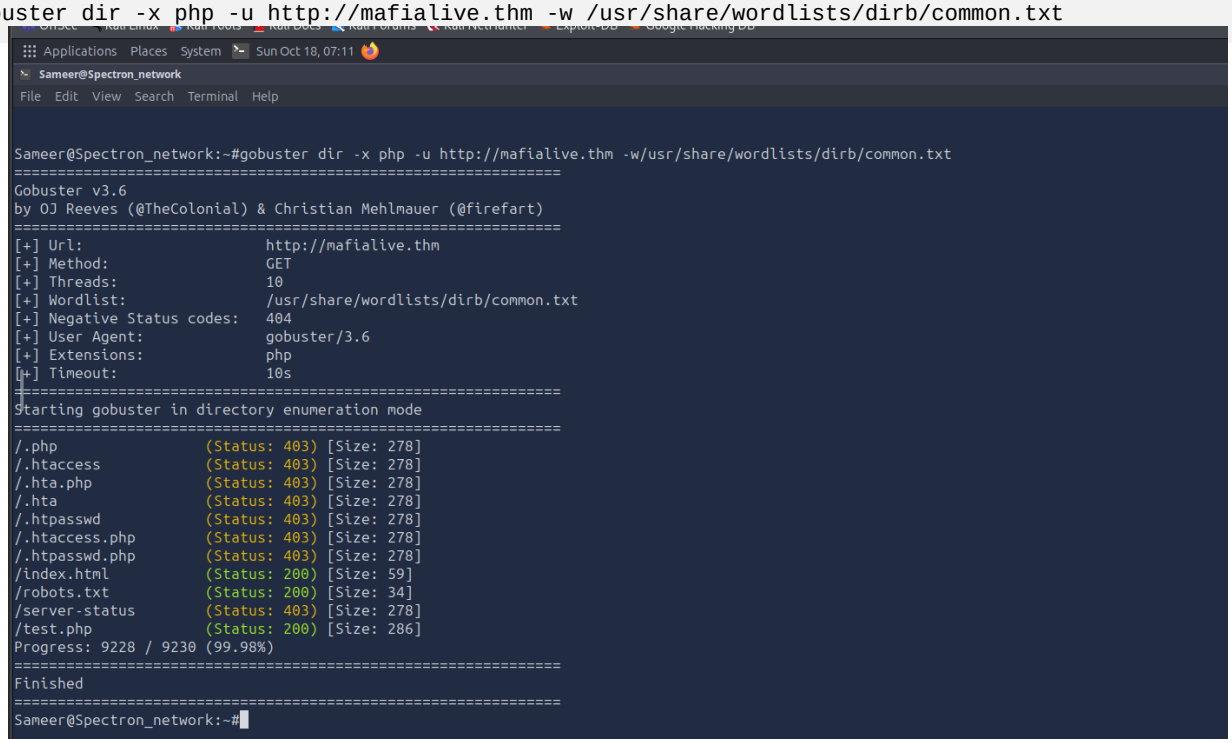


Figure 5 – Gobuster directory enumeration discovering test.php (HTTP 200).

4.3 Phase 3a – Local File Inclusion & Source Code Disclosure

Affected URL: `http://mafialive.thm/test.php`

Severity: **High** | CVSS (approx.): 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

The `view` GET parameter on `test.php` is passed directly into a PHP `include()` call. Server-side source code (recovered in a later step) showed the only control in place rejects requests containing the literal string `../..` and otherwise requires the path to simply contain `/var/www/html/development_testing`. This is a weak allow-list: an absolute path pointing anywhere under that string – including sibling files the developers did not intend to expose – is accepted without restricting which file is actually read.

Step 1 – Direct inclusion of an out-of-scope developer file

A direct absolute path to a leftover developer test file (`mrrobot.php`) inside the whitelisted directory was supplied, confirming the LFI executes/returns the target file's rendered output.

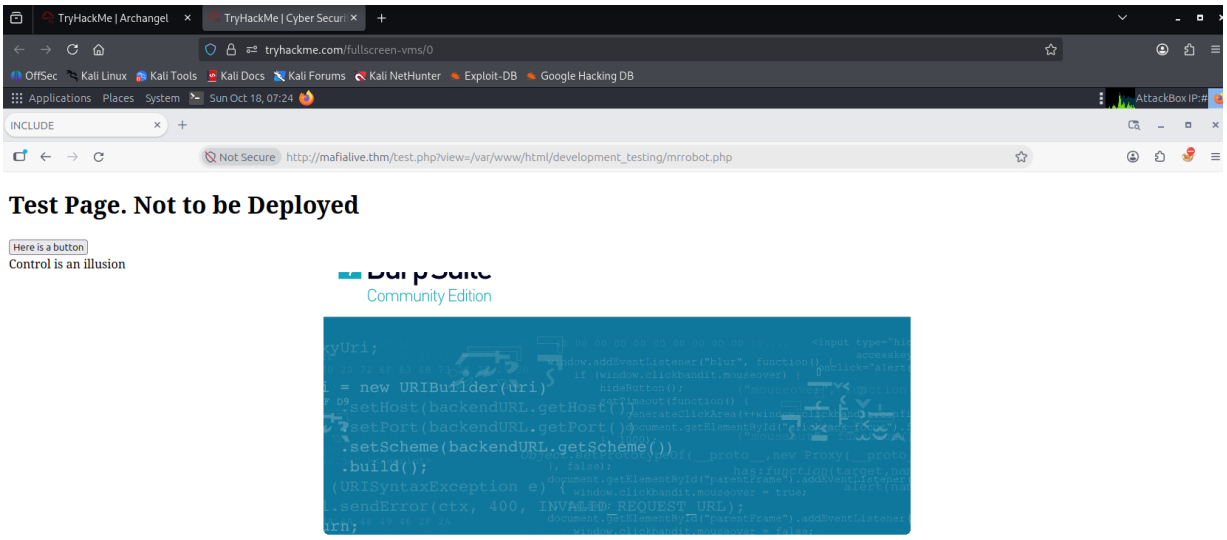


Figure 6 – test.php?view=/var/www/html/development_testing/mrrobot.php rendering the include()'d file's output directly in the browser.

Step 2 – Source code disclosure via php://filter

To read the PHP source itself (rather than its executed output), the `php://filter/convert.base64-encode/resource=` wrapper was chained with the target path. This causes PHP to base64-encode the file before inclusion instead of executing it, returning the raw source as a safe, printable blob.

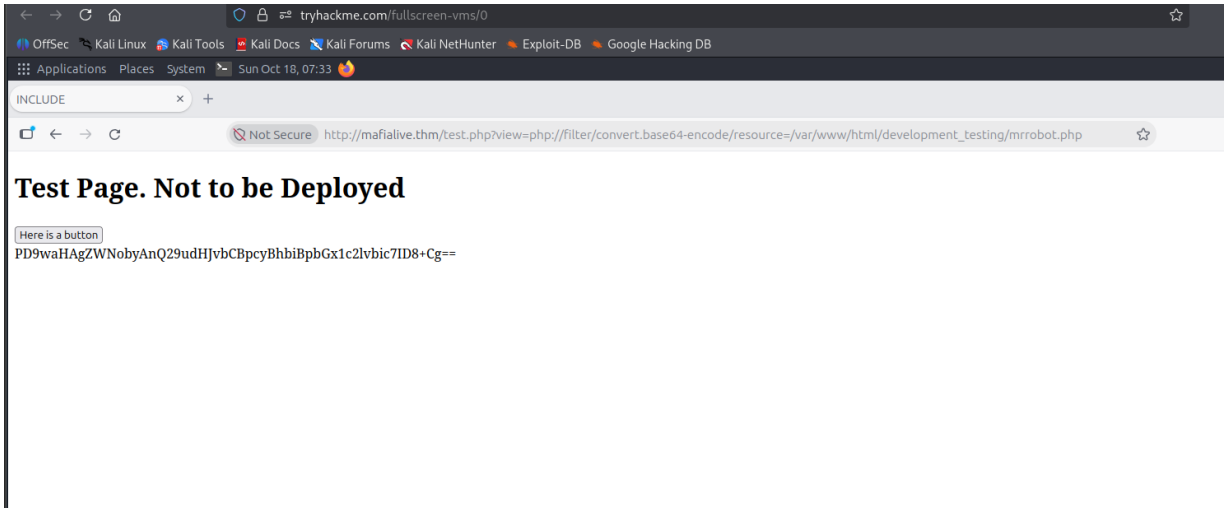
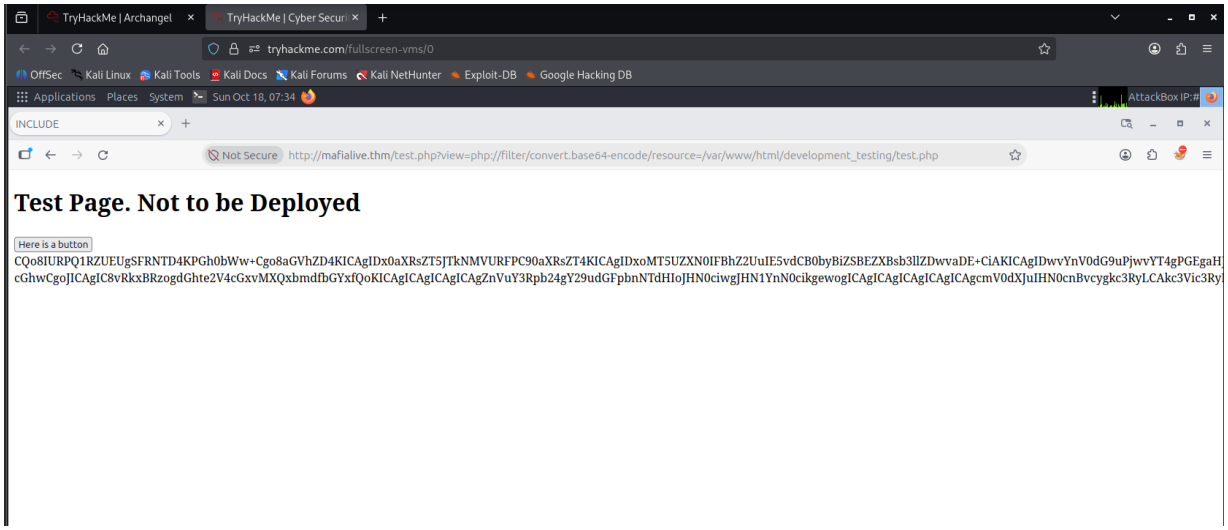


Figure 7 – Base64-encoded source of mrrobot.php returned via the php://filter wrapper.


TryHackMe | Archangel
TryHackMe | Cyber Secur
tryhackme.com/fullscreen-vms/0
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Applications Places System Sun Oct 18, 07:34
AttackBox IP:#
INCLUDE
Not Secure http://mafialive.thm/test.php?view=php://filter/convert.base64-encode/resource=/var/www/html/development_testing/test.php
Test Page. Not to be Deployed
Here is a button
CQo8IURPQ1RZUEUgSFRNTD4KPGh0bWw+Cgo8aGVhZD4KICAgIDx0aXRsZT5jTkNMVURFPC90aXRsZT4KICAgIDxoMT5UZXN0IFBhZ2UuIE5vdCB0byBiZSBEZXBsb3llZDwvaDE+CiAgICAgPnV0dG9uPjwvYT4gPGEgICAgIGhwCgoJICAgIC8vRkxBRzogdGhte2V4cGxvaXMXQxbmdfbGYxfQoKICAgICAgICAg

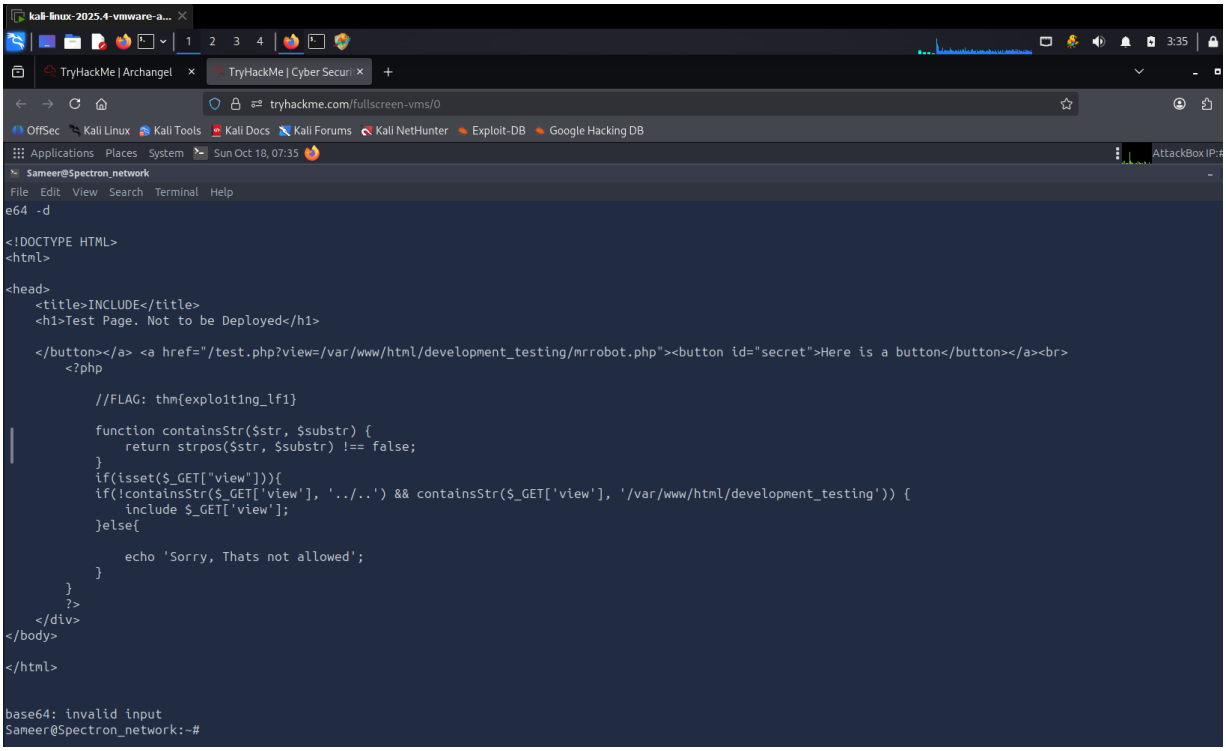


Figure 9 – Decoded PHP source of test.php showing the flawed containsStr() whitelist check and the embedded flag.

FLAG 2: thm{explo1t1ng_lf1}

Step 4 – Pivoting the LFI beyond the web root

Because the validation check only tests whether the supplied string *contains* the whitelisted directory – without canonicalising the path – a mixed path-traversal payload (`..%2f..%2f` style segments combined with the required substring) was used to walk out of the web root and read the Apache access log directly.

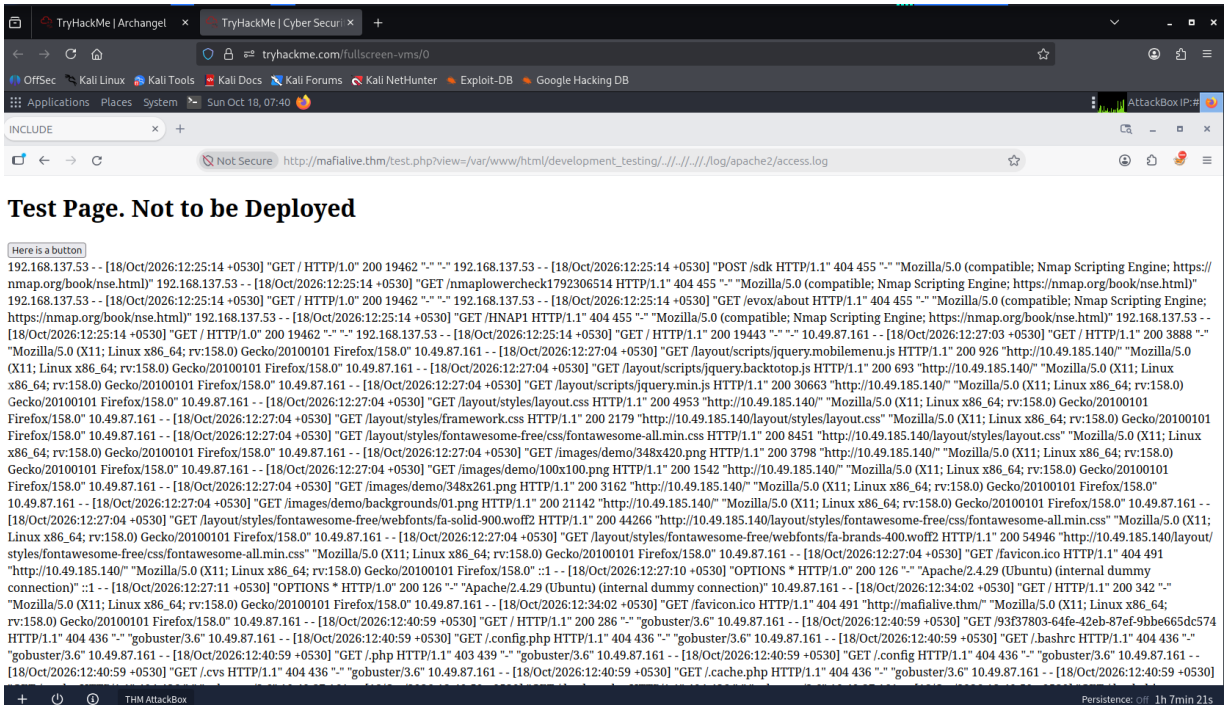


Figure 10 – apache2/access.log read through the LFI, confirming file access outside the web root and exposing request logs (including the tester's own reconnaissance traffic) that can be used for log-poisoning RCE.

Significance: Apache logs every request verbatim, including the User-Agent header. Because this log file is now confirmed readable – and therefore include()-able – via the LFI, sending a single HTTP request with a PHP payload in the User-Agent header (e.g. `<?php system($_GET['cmd']); ?>`) and then including `access.log` through `test.php?view=` would cause the PHP interpreter to execute that payload, yielding remote command execution. This is the standard LFI-to-RCE log-poisoning technique and is the next planned step in this engagement (see Section 6).

5. Captured Flags

#	Flag	Captured Via
1	thm{f0und_th3_r1ght_h0st_n4m3}	Virtual host discovery (mafialive.thm)
2	thm{explo1t1ng_lf1}	Source code disclosure of test.php via LFI + php://filter

6. Remaining Attack Path (Planned / Pending Evidence)

The project problem statement requires proof-of-concept evidence for every phase of the Technical Execution Lifecycle. At the time this report was compiled, screenshots were available only through the LFI source-disclosure and log-access steps above. The remaining phases are scoped below, based on the attack surface already confirmed, so they can be executed and the resulting evidence appended before final submission.

6.1 Phase 3b – Log Poisoning to Web Shell

- Intercept a request to mafialive.thm in Burp Suite and set the User-Agent header to a PHP web-shell one-liner, e.g. `<?php system($_GET['cmd']); ?>`.
- Send the request so the payload is written verbatim into apache2/access.log.
- Call `test.php?view=/var/www/html/development_testing/.../.../log/apache2/access.log&cmd=id` to trigger execution and confirm command output in the response.
- Capture a screenshot of the executed command output as Phase 3 completion evidence.

6.2 Phase 4 – Metasploit & Meterpreter Upgrade

- Stage a Meterpreter payload (e.g. `php/meterpreter/reverse_tcp`) and set up `exploit/multi/handler` in msfconsole with a matching LHOST/LPORT.
- Use the established log-poisoning RCE to fetch and execute the staged payload on the target.
- Confirm an interactive Meterpreter session opens in msfconsole; capture the session banner as evidence.

6.3 Phase 5 – Linux Local Privilege Escalation

- From the Meterpreter/shell session, enumerate SUID binaries (`find / -perm -4000 -type f 2>/dev/null`), sudo rights, and writable cron/service files.
- Identify and exploit the misconfigured SUID binary associated with the Archangel room to obtain a root shell.
- Capture `id` / `whoami` output confirming root access as final evidence.

7. Risk Rating Summary

Based on findings confirmed to date, the overall risk exposure of the Spectron Network web application is rated **HIGH**. The LFI vulnerability alone provides an unauthenticated attacker with arbitrary file read, source-code disclosure, and a direct path to remote code execution without any credentials or user interaction required.

Finding	Severity	CVSS (approx.)
Local File Inclusion (test.php?view=)	High	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
PHP Source Code Disclosure via php://filter	Medium	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Sensitive log file accessible outside web root	Medium	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)

8. Defensive Remediation & Hardening Plan

8.1 Code Fix – Secure File Inclusion Handling

Replace the substring-based allow-list with a canonical-path whitelist that resolves symlinks and relative segments before comparison, and restrict inclusion to a fixed, enumerated set of filenames rather than an arbitrary user-supplied path:

```
<?php
$allowed = ['home' => 'home.php', 'about' => 'about.php']; // explicit whitelist, no user path
$page = $_GET['view'] ?? 'home';
if (array_key_exists($page, $allowed)) {
  include __DIR__ . '/pages/' . $allowed[$page];
} else {
  http_response_code(400);
  echo 'Invalid page requested';
}
?>
```

Never pass user input directly into include()/require(). If dynamic inclusion is unavoidable, use `realpath()` and verify the resolved path's prefix matches an approved base directory before including it, and reject any path containing null bytes, wrapper schemes (`php://`, `data://`, `expect://`), or traversal sequences in any encoding.

8.2 Server Hardening

- Set `allow_url_include = Off` and `allow_url_fopen = Off` in php.ini to prevent remote file inclusion and reduce wrapper abuse.
- Restrict Apache/PHP's effective filesystem permissions with `open_basedir` so the web process cannot read logs or files outside its own web root.
- Set log directories (e.g. `/var/log/apache2/`) to be unreadable by the web-server user, or relocate logs outside any path reachable via application-controlled includes.
- Remove development/test artifacts (e.g. `mrrobot.php`, `development_testing/`) from production deployments entirely.
- Disable directory listing and audit for other leftover debug endpoints using the same gobuster wordlist approach used in this assessment.
- Apply least-privilege file permissions and remove any unnecessary SUID bits on system binaries to limit post-exploitation privilege escalation if RCE is ever achieved.

9. Appendix A – Risk Definitions (CVSS)

This report uses CVSS v3.1 base scores to determine the severity of each finding. The Common Vulnerability Scoring System provides a numerical (0–10) representation of severity, derived from Exploitability, Scope, and Impact sub-metrics.

CVSS Score Range	Severity Category
0.0	Informational / Observational
0.1 to 3.9	Low
4.0 to 6.9	Medium
7.0 to 8.9	High
9.0 to 10.0	Critical

10. Appendix B – Tool Usage

Tool	Purpose
Nmap	Port and service discovery against the target host.
Gobuster	Directory and file enumeration on the web application.
Burp Suite Community/Professional	Intercepting proxy for manual request crafting (planned for log poisoning in Phase 3b).
Firefox (THM AttackBox)	Manual browser-based exploitation of the LFI vulnerability.
base64 (coreutils)	Local decoding of source code disclosed via the php://filter wrapper.
Metasploit Framework (planned)	msfconsole / exploit multi-handler for Meterpreter session establishment (Phase 4).
Kali Linux toolset	General-purpose exploitation toolkit available on the attack host.

11. Disclaimer

This assessment was performed exclusively within the TryHackMe “Archangel” lab environment for academic and training purposes as part of the Cybersecurity Major Project. All IP addresses, hostnames, and the “Spectron Network” enterprise alias used throughout this document refer solely to the assigned lab target and attacking Kali host, and do not represent any real production organization. No activity described in this report was performed against any system outside the authorized lab scope.

It should be noted that it is not possible to completely guarantee the security of any network, system, or application, and this report does not constitute a guarantee of security for the tested environment. Sections marked “Pending Evidence” describe the planned and technically justified continuation of the attack chain based on findings already confirmed, and should be completed with corresponding screenshots prior to final submission.